

# inicio

June 24, 2026

Introduccion

June 22, 2026



1 01PAO26-26 - Python 01



Kevin Xavier Suasnavas Villareal

Link a GitHub

2 Introducción a Python

2.0.1 Principales características

Lenguaje de Propósito general

Interpretado, no compilado. Más flexible y portable

Tipado dinámico

Fuertemente tipado

Énfasis en la legibilidad

Lenguaje de alto nivel

Gestión automática de memoria. Cuando el rendimiento es crítico, hay lenguajes más apropiados.

Multiparadigma: orientado a objetos, procedural y funcional

Indentación para eliminar bloques de código

Gran librería con módulos para múltiples tareas.

Multiplataforma

2.0.2 ¿Por qué Python?

“Python is used in pursuits as diverse as data science, film-making, computer science education, IT management, and much more. There really is no computing field that Python has not touched (except maybe kernel development).

Python is loved for its flexibility, beautiful and succinct syntax, object-oriented purity, and bustling community.

The strong community is important because it means Python is welcoming to newcomers and has a large ecosystem of available libraries for developers to build upon.”

Kopec, D. (2019). Classic Computer Science Problems in Python. Simon and Schuster.

Uno de los lenguajes más usados en todo el mundo.

Perfecto para introducción a la programación.

Ecosistema amplio con librerías estables para múltiples áreas.

Comunidad muy participativa y mucha documentación.

Incrementa la productividad del desarrollador.

- Menos programación. Código más compacto.
- No largas compilaciones.
- Código legible, fácilmente mantenible.

Fácil integración con herramientas y otros lenguajes.

Versátil en la tipología de programación.

Es multiplataforma, portable.

Compilable para mejorar ejecución.

Trabajo de memoria con gran cantidad de datos. Stack Overflow Developer Survey 2023 The Top Programming Languages 2024 Tiobe Index HackerRank Developer Skills Report

### 2.0.3 ¿Qué puedo hacer con Python?

Herramientas shell (administración de sistemas)

Manipulación de ficheros

Ejecución de comandos

Desarrollo de interfaces Gráficas de Usuario (GUIs)

Internet and network communication.

- Generación y parseado de XML y JSON
- Recuperación de webs por URL
- Comunicación a través de sockets
- Transferencia de ficheros por FTP

Programación de base de datos

Computación numérica (NumPy)

Análisis de lenguaje natural

Aprendizaje automático e inteligencia artificial

Programación multimedia

Tratamiento de datos.

- linear algebra
- statistical modeling
- visualization
- computational linguistics
- graph analysis
- machine learning
- business intelligence
- data storage and retrieval

#### 2.0.4 Filosofía Python

```
[5]: import this
```

The Zen of Python, by Tim Peters

```
Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
```

#### 2.0.5 Documentación

Python Enhancement Proposals (PEP)

- Index
- Purpose and Guidelines

Python Style Guide

### 3 Intérprete de Python y ejecución de scripts

#### 3.0.1 Antes de empezar

Se puede utilizar diferentes formas para ejecutar código:

Línea de comandos o terminal, Shell de Python o interactivo, python/ipython

IDE -> Eclipse, Pycharm, Sublime, Nano, VSCode, Atom, Spyder, ...

Google Colab, Azure Notebooks, Jupyter

### 3.0.2 ¿Qué es un intérprete?

Un programa que ejecuta otros programas.

Una capa de software entre tu código y el hardware que lo ejecuta.

Debe estar instalado en tu ordenador para poder ejecutar código Python.

Para la especificación de Python , existen varias implementaciones:

- CPython (implementación en C). Es el más común.
- Jython (implementación en Java).
- IronPython (implementación en .NET).

### 3.0.3 ¿Cómo se lleva a cabo la ejecución de scripts?

```
[6]: #Imprimir en pantalla hello"  
print('Hello World')
```

Hello World

Guía con detalles para Windows, Linux y macOS: Real Python

### 3.0.4 Perspectiva del desarrollador

Un script en Python es un fichero de texto que:

- Contiene instrucciones Python.
- Tiene extensión .py.

Puedes ejecutar scripts:

- Línea de comandos.
- IDE.

### 3.0.5 Perspectiva de Python

Compilación de código fuente a byte code.

Código byte code se ejecuta más rápido.

Ficheros .pyc que se almacenan en caché.

Permite saltarse el paso de compilación.

Python Virtual Machine (PVM)

Ejecuta las instrucciones en byte code.

### 3.0.6 ¿Cómo puedes ejecutar tus scripts?

### 3.0.7 Línea de comandos

Ejecutar “py” o “python” para abrir una sesión interactiva del intérprete.

También ejecutando la aplicación “Python” desde el menú inicio.

Los caracteres “»>” indican que estás en una sesión interactiva.

Útil para experimentación y testing.

```
[7]: print('Hello world')
```

```
Hello world
```

Inconveniente: los programas que ejecutas en la línea de comandos desaparecen tras ser ejecutados.

### 3.0.8 REPL

Sistema interactivo para comunicarse con el ordenador en un lenguaje, Python.

Se debe cumplir:

- Read. El ordenador pueda leer unidades como entrada.
  - Evaluate. El código pueda ser procesado.
  - \* Print. Los resultados puedan verse.
  - Loop. Continuar con la conversación.

### 3.0.9 Ficheros

Permite almacenar programas.

Ficheros de texto con instrucciones Python.

– No olvidar shebang en Linux → `#!/usr/bin/env python3`

Terminología (varía según fuentes):

- Scripts o programas: programa principal.
- Módulos: ficheros importados desde otros ficheros.

Se puede lanzar pasando nombre de fichero a comando python.

```
[9]: print("> python ./script1.py")  
      print("Hello world")
```

```
> python ./script1.py  
Hello world
```

Otra alternativa (a partir de Python 3.3) es:

```
[10]: print("> py ./script1.py")  
       print("Hello world")
```

```
> py ./script1.py  
Hello world
```

O incluso:

```
[11]: print("> ./script1.py")
      print("Hello world")
```

```
> ./script1.py
Hello world
```

También es posible hacer doble-click sobre fichero .py.

3.0.10 Instalación de librerías

pip (built-in > Python 3.4)

pipenv (gestiona paquetes y entornos virtuales) o virtualenv

3.0.11 Jupyter

```
[15]: !pip install pandas
```

```
Requirement already satisfied: pandas in
C:\Users\Kevin\anaconda3\envs\machine2026\Lib\site-packages (3.0.3)
Requirement already satisfied: numpy>=1.26.0 in
C:\Users\Kevin\anaconda3\envs\machine2026\Lib\site-packages (from pandas)
(2.4.6)
Requirement already satisfied: python-dateutil>=2.8.2 in
C:\Users\Kevin\anaconda3\envs\machine2026\Lib\site-packages (from pandas)
(2.9.0.post0)
Requirement already satisfied: tzdata in
C:\Users\Kevin\anaconda3\envs\machine2026\Lib\site-packages (from pandas)
(2026.2)
Requirement already satisfied: six>=1.5 in
C:\Users\Kevin\anaconda3\envs\machine2026\Lib\site-packages (from python-
dateutil>=2.8.2->pandas) (1.17.0)
```

```
[16]: # Preguntar de forma interactiva
      # print?

      # Usar shift + tab para hint con ayuda

      # Comentar una linea

      # """
      # Se puede comentar un texto
      # más grande para hacer descripciones detalladas con más de una línea
      # """

      import pandas as pd

      print("Siempre podremos 'poner' los comentarios en forma de salida, para ver_
      ↪ resultados")
```

```
print('Siempre podremos "poner" los comentarios en forma de salida, para ver_
↪resultados')

universidad = 'ITQ'

print(universidad)
```

Siempre podremos 'poner' los comentarios en forma de salida, para ver resultados  
 Siempre podremos "poner" los comentarios en forma de salida, para ver resultados  
 ITQ

Celdas Python

- Marca el número de la ejecución
  - Marca si está en ejecución con \*

Kernel

- Se puede resetear el kernel cuando haya problemas
  - Limpiar y resetear kernel

Comandos especiales ( IPython )

- %run: ejecuta un script

%time: ejecuta una línea de código y devuelve el tiempo de ejecución.

%%time: ejecuta una celda de código y devuelve el tiempo de ejecución.

!: ejecuta un comando de consola.

%%bash: ejecuta un comando en bash en un subprocesso.

%%bash

ls -la

%%js: ejecuta JavaScript.

```
[19]: %%js

var master = "Máster en Inteligencia Artificial";
var codigo = 1;

alert("Esta asignatura tiene el código: " + codigo);
```

<IPython.core.display.Javascript object>

Atajos de teclado (shortcuts): Documentación de Jupyter

```
[20]: !dir
```

El volumen de la unidad C no tiene etiqueta.

El número de serie del volumen es: 18C6-4BF1

Directorio de C:\Users\Kevin\Desktop\machine\practicas

```

24/06/2026  22:36    <DIR>          .
24/06/2026  11:36    <DIR>          ..
24/06/2026  11:37    <DIR>          .ipynb_checkpoints
24/06/2026  11:44    <DIR>          img
24/06/2026  22:36                31.749 inicio.ipynb
                        1 archivos          31.749 bytes
                        4 dirs  22.948.872.192 bytes libres

```

```

[21]: %%js
      var asignatura = "Materia de Machine E-learning 1";
      var codigo = 1;
      alert("Esta asignatura tiene el código: " + codigo);

```

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

3.0.12 Performance Sample, NumPy vs Plain Python

```

[22]: import numpy as np

my_arr = np.arange(10000000) # Utilizando NumPy Arrays
my_list = list(range(10000000)) # Utilizando Listas

print(my_list)

```

IOPub data rate exceeded.

The Jupyter server will temporarily stop sending output to the client in order to avoid crashing it.

To change this limit, set the config variable

`--ServerApp.iopub\_data\_rate\_limit`.

Current values:

ServerApp.iopub\_data\_rate\_limit=1000000.0 (bytes/sec)

ServerApp.rate\_limit\_window=3.0 (secs)

```

[23]: %time for _ in range(10): my_arr2 = my_arr * 2
      %time for _ in range(10): my_list2 = [x * 2 for x in my_list]

```

CPU times: total: 516 ms

Wall time: 552 ms

CPU times: total: 10.9 s

Wall time: 11.6 s

4 Referencias

González Duque, R. Python para todos. Licencia Creative Commons.

Boschetti, A. y Massaron, L. (2016). Python Data Science Essentials, Second Edition. Birmingham - Mumbai: Packt.



Python Software Foundation. Built-in Functions — Python Documentation. Recuperado el 16 de noviembre de 2018 de Python Documentation

Tutorialspoint. Python Tutorial. Recuperado el 16 de noviembre de 2018 de Tutorialspoint

Kenneth Reitz (2018). Code Style — The Hitchhiker’s Guide to Python. Recuperado el 16 de noviembre de 2018 de The Hitchhiker’s Guide to Python

Frank Hofmann (2018). Introduction to the Python Coding Style. Recuperado el 16 de noviembre de 2018 de Stack Abuse

Perez, Fernando (2017). Design At Large: Fernando Perez: The Architecture of Jupyter. YouTube

[ ]: